DBMS 的优势与性能比较

姓名：易楚曦　　学号：12510903

GitHub 项目

github.com/luanmenglei/dbh_project1

1 结果

1. 相比直接操作文件，DBMS 的独特优势是什么?

DBMS 提供现成的关联约束、事务回滚和查询功能，减少应用程序自行实现和维护这些功能的工作。

实验验证了以下能力：

- **引用约束：** 两库都拒绝插入引用不存在电影的演职记录，避免产生无效关联。
- **事务回滚：** 上述插入失败后，回滚也撤销了事务中先前完成的片长修改。
- **索引支持：** 两库提供现成的索引与查询机制；本实验中，添加人物索引显著缩短了关联查询耗时。

这些能力也可以用文件和代码自行实现，但需要额外维护。实际上，在本实验中，已装载索引的文件程序按编号读取单条记录比使用数据库更快。

2. PostgreSQL 和 openGauss 哪个更好? 按什么标准?

综合本实验的耗时、每秒完成的操作数和 P95 延迟，PostgreSQL 整体表现更好。

表 1　两库代表性结果；完整指标见后文

指标与条件	PostgreSQL	openGauss	本次表现
100× 标题子串计数耗时	128.13 ms	322.35 ms	PostgreSQL 更快
100× 固定返回 54 行的连接耗时	1.42 ms	1.07 ms	openGauss 更快
10×、8 客户端混合负载吞吐量	9.43 万次/秒	5.50 万次/秒	PostgreSQL 更高

2 实验

实验采用课程电影数据 `filmdb.sql`，包含国家、电影、人物和演职关系四张表。 后三表扩展为原来的 $1, 10, 100$ 倍，并平移编号以保持关联关系。 复制后的数据保留原字段分布，规模见 表 2。

表 2　数据规模与记录数；100 倍 TSV 合计约 161.1 MiB

规模	电影	人物	演职关系
1×	9,204	16,489	46,850
10×	92,040	164,890	468,500
100×	920,400	1,648,900	4,685,000

测试使用 PostgreSQL 14.24（下称 PG）和 openGauss 6.0.1（下称 OG），运行于 Ubuntu 22.04 服务器，硬件为双路 AMD EPYC 7542 和 NVMe 固态硬盘。 两库轮流运行，各限 8 GiB 内存；文件程序、客户端与脚本共用 4 GiB。 关闭单条查询内部的并行执行，基础配置将数据库与客户端分别绑定 8 核与 2 核。

实现。 主程序使用 C++17，数据库端通过 libpq 执行 SQL，文件端读取 TSV（以制表符分隔的文本文件）。 标题搜索区分大小写，两端分别使用 `strpos(title,$1)>0` 和 `string::find`，返回计数或电影编号与标题。 姓名更新均将名字和姓氏中的大写 T 替换为 Ttt；数据库提交事务，文件端重写临时文件、同步落盘后替换原文件。

测量与核验。 正式计时前先运行查询，使缓存进入稳定状态，即“预热”。串行查询预热一次；并发预热 5 秒，再测量 20 秒。通常每组测三轮，报告中位数及最小值至最大值。 计时包含查询执行、结果接收及校验值计算，不含数据导入及连接准备；下文称为端到端耗时。三个实现通过结果校验值（内容摘要）核对查询与更新结果。每轮更新前恢复原数据，两库开启同步提交与 `fsync`，保证提交时同步落盘。

表 3　实验设计概览

实验	比较方法
标题搜索	搜索标题包含 The、Love、Star 的电影，比较计数与返回编号、标题的耗时。
批量更新	将姓名中的 T 替换为 Ttt，比较文件重写与数据库更新的耗时。
单条记录查询	每轮随机查 100 个编号，比较文件扫描、文件索引及两库主键查询；另做每轮十万次补测。
人物关联查询	按人物编号连接演职关系表与电影表；三个规模均固定返回 54 行。
同库索引对照	固定数据与查询，五轮交替增删数据库中的人物索引，比较有、无索引的耗时。
分组聚合	按国家和年份统计电影数、非空片长总和及数量。
并发只读	将客户端数从 1 增加到 32，测量吞吐量与 P95 延迟。
CPU 资源干预	分别增加客户端与数据库绑定的 CPU 核数，观察吞吐变化。
混合读写	10× 数据、8 个客户端下，比较只读与约 90% 读、10% 同步写的负载。
外键与回滚	先修改片长，再插入非法演职记录；两库拒绝插入，回滚后片长及四表内容摘要恢复。

2.1 访问方式对查询性能的影响

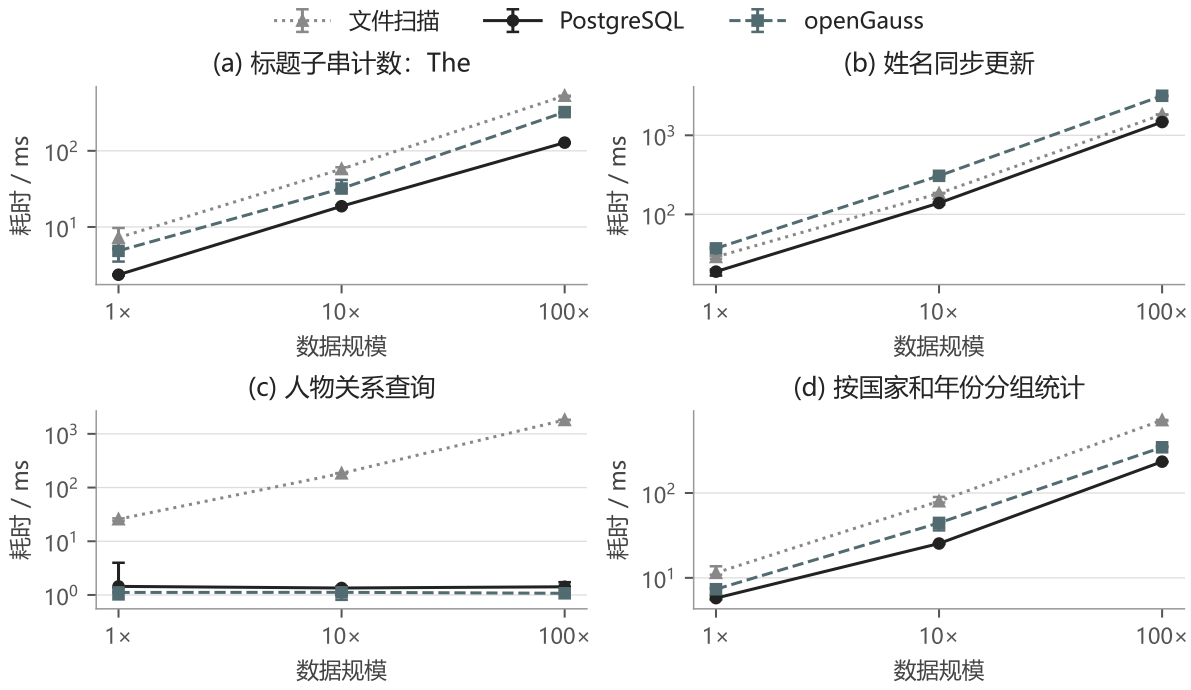


图 1　数据规模与四类操作的耗时。横纵轴均为对数刻度，各面板纵轴范围不同；点为三轮中位数，误差线为最小值至最大值。人物连接在三个规模下均返回 54 行。

人物关联查询中，文件程序依次扫描演职关系表与电影表；数据库先经人物索引定位演职记录，再用电影主键读取详情。100 倍下文件端约 1814.34 ms，数据库索引连接约 1 ms。三个规模下均为 openGauss 略快于 PostgreSQL。

为单独验证索引的作用，另在 100 倍数据下固定同一查询，五轮交替增删数据库中的人物索引，每次预热后计时，导入与建索引不计入耗时。 PG 的有、无索引耗时中位数分别为 1.43、345.48 ms，OG 为 1.24、703.52 ms；逐轮加速比的中位数分别为 243.3、566.9 倍。这说明人物索引是该查询高效的关键；这组补测与图中的原实验分开统计。

标题计数和分组统计均需遍历电影表。100 倍下，PG 分组统计耗时为文件端的约三分之一。

2.2 已装载索引下的单条记录读取

按编号查询单条记录称为“点查”。文件偏移索引记录每个编号对应的文件位置，查询时直接跳到该位置读取；数据库使用主键查找。表 4 中，文件扫描平均单次约 263 ms，已装载索引的文件查询约 4.84 µs。

表 4　短批次主键点查：100 倍数据，每轮 100 次请求

后端 / 访问路径	平均单次 / µs	三轮范围 / µs
文件扫描	262765.74	[262640.25, 264006.09]
文件偏移索引	4.84	[4.67, 4.95]
PostgreSQL	97.43	[92.14, 98.34]
openGauss	194.43	[137.52, 195.71]

三轮使用同一随机序列，报告每轮平均单次耗时的中位数及范围。计时包含逐次随机选择编号和转换字符串，不含候选编号读取、连接及索引准备；文件索引建立另需 0.946 s（单次测量）。

2.3 扫描、更新与混合负载比较

在预热后的全表子串搜索、分组聚合及批量字符串替换中，100 倍规模下 PG 的端到端耗时均低于 OG。

表 5　四类串行操作的耗时：100 倍数据

操作	文件 / ms	PG / ms	OG / ms
The 子串计数	529.23 [515.62, 529.85]	128.13 [126.06, 128.68]	322.35 [312.23, 324.21]
姓名同步更新	1827.72 [1819.72, 1836.73]	1472.21 [1471.99, 1481.63]	3162.42 [3132.07, 3169.66]
人物连接	1814.34 [1811.70, 1816.68]	1.42 [1.39, 1.73]	1.07 [1.06, 1.09]
国家 / 年份聚合	722.42 [703.17, 726.08]	234.02 [230.59, 234.67]	345.67 [345.26, 356.47]

三轮操作耗时的中位数，下行为［最小值，最大值］。 本表与混合负载来自第一批实验，客户端 CPU 绑定与 小节 2.4 不同，两批分别比较。

2.3.1 标题搜索与结果集大小

表 6　返回电影编号与标题的搜索：100 倍数据，三轮耗时中位数

关键词	返回行数	文件 / ms	PG / ms	OG / ms
The	156,300	554.47	209.60	402.67
Love	8,800	528.57	125.68	313.70
Star	5,000	524.14	125.15	310.20

表 6 的三组查询都扫描电影表，PG 均较快。PG 的 The 返回记录耗时 209.60 ms、仅计数 128.13 ms，说明接收和处理结果也有成本。Love 和 Star 返回行数相差 3,800 行而耗时接近。

2.3.2 批量更新的端到端耗时

把姓名中的大写 T 替换成 Ttt，100 倍数据共修改 128,700 行。文件重写整张人物文件，同步后替换原文件； 数据库更新匹配记录并提交事务。表 5 中 PG 最快，OG 慢于文件端。

2.3.3 读多写少负载的吞吐量与尾延迟

表 7　只读与混合负载：10 倍数据，8 个客户端

负载与指标	PostgreSQL	openGauss
只读吞吐（万次/秒）	9.41	7.35
混合吞吐（万次/秒）	9.43	5.50
只读负载：读 P95 / ms	0.106	0.126
混合负载：读 P95 / ms	0.121	0.218
混合负载：写 P95 / ms	0.180	0.441

各项为三轮中位数。P95 先逐轮分别统计读、写请求，再取中位数，未合并样本。 混合负载约 90% 为主键读取、10% 为单行片长更新，每次更新单独同步提交；属第一批实验。

表 7 显示，加入少量同步写后，两库的读 P95 都上升，OG 的混合吞吐下降更明显。 PG 的两种负载吞吐接近，但读 P95 仍由 0.106 ms 升至 0.121 ms，说明少量写入也会影响读请求的延迟。

2.4 并发扩展与 CPU 瓶颈分析

在数据库绑定 8 核、客户端绑定 2 核的并发只读下，增加可用 CPU 核数的实验显示，高并发时 PostgreSQL 的吞吐主要受客户端 CPU 限制，openGauss 主要受数据库 CPU 限制。

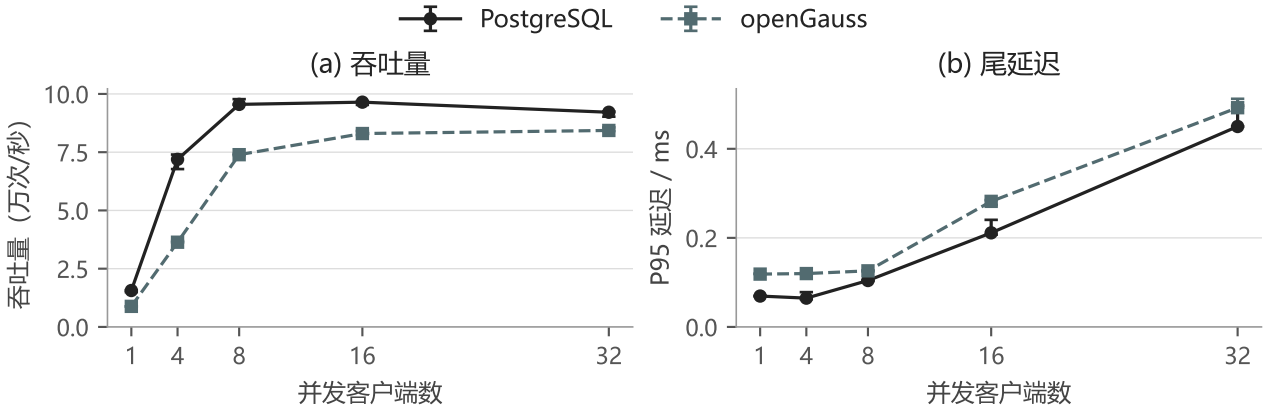


图 2　并发只读扩展：10 倍数据，数据库绑定 8 核、客户端绑定 2 核。点为三轮中位数，误差线为最小值至最大值。每个客户端收到结果后再发下一次请求；P95 先逐轮计算，再取中位数。

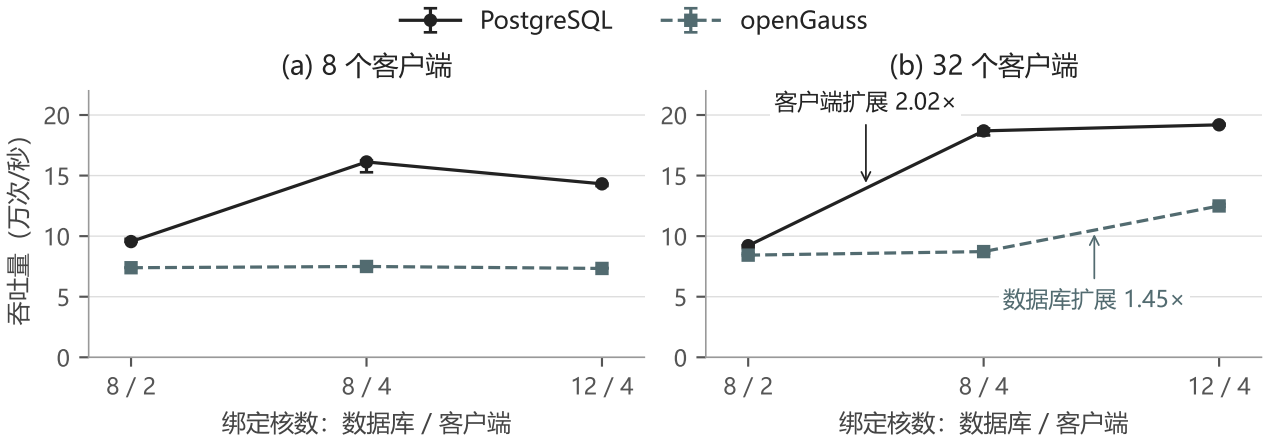


图 3　CPU 资源干预：10 倍数据，与 图 2 同批；各条件查询与数据库设置相同。点为三轮中位数，误差线为最小值至最大值，两面板纵轴一致。标注倍数为逐轮配对比值的中位数。

图 2 中，并发从 1 增加到 16 时 PG 吞吐上升；高并发时吞吐趋于平台，P95 延迟随之上升。 32 客户端纯读下，PG 客户端用了 2.00 核、数据库仅 4.06 核； OG 客户端 1.88 核、数据库 7.99 核，接近 8 核上限。 这里的核数是平均 CPU 用量，表示相当于多少个核心持续工作；由每 0.5 秒采样估算，剔除启动与预热，其窗口与吞吐测量不完全重合。

图 3 的 32 客户端组中，客户端从 2 核增至 4 核，PG 吞吐由 9.21 升至 18.69 万次/秒，OG 变化较小；随后数据库从 8 核增至 12 核，OG 由 8.72 升至 12.49 万次/秒，PG 仅小幅上升。 延长至预热 10 秒、测量 120 秒的单轮确认得到相同趋势，详细记录见仓库。